VISVESVARAYA TECHNOLOGICAL UNIVERSITY
"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

OPERATING SYSTEMS

Submitted by

Bhoomika (1BM25CS428)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by Bhoomika (1BM25CS428), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Prof. Lakshmi Neelima
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

LAB PROGRAM – 1:

Question:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS
b) SJF (non-pre-emptive)
c) SRTF (SJF pre-emptive)
d) Priority (non-pre-emptive)
e) Priority (pre-emptive)
f) Round Robin

a) FCFS:

CODE:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   int n, i;
   printf("\n\tNAME : R Nandini\tUSN : 1BM25CS497\tSECTION : 'O'\n\n");
printf("Enter number of processes: ");     scanf("%d", &n);

   int at[n], bt[n], ct[n], tat[n], wt[n], rt[n];
float avg_tat = 0, avg_wt = 0, avg_rt = 0;

   printf("Enter Arrival Time and Burst Time:\n");
for(i = 0; i < n; i++)
   {
      printf("P%d: ", i+1);
scanf("%d %d", &at[i], &bt[i]);
   }

   ct[0] = at[0] + bt[0];

   for(i = 1; i < n; i++)
   {
```

```
        if(ct[i-1] < at[i])
ct[i] = at[i] + bt[i];
else
            ct[i] = ct[i-1] + bt[i];
    }

    for(i = 0; i < n; i++)
    {
        tat[i] = ct[i] - at[i];         wt[i] =
tat[i] - bt[i];         rt[i] = wt[i];   //
FCFS: RT = WT

        avg_tat += tat[i];
avg_wt += wt[i];         avg_rt
+= rt[i];
    }

    avg_tat /= n;
avg_wt /= n;     avg_rt
/= n;

    printf("\nProcess\tAT\tBT\tTAT\tWT\tRT\n");
for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], tat[i], wt[i], rt[i]);
    }

    printf("\nAverage Turnaround Time = %.2f", avg_tat);
printf("\nAverage  Waiting  Time  =  %.2f",  avg_wt);
printf("\nAverage  Response  Time  =  %.2f",  avg_rt);
return 0;
}
```

OUTPUT:

```
Enter number of processes: 4
Enter Arrival Time and Burst Time:
P1: 0 2
P2: 1 2
P3: 5 3
P4: 6 4

Process AT      BT      TAT     WT      RT
P1      0       2       2       0       0
P2      1       2       3       1       1
P3      5       3       3       0       0
P4      6       4       6       2       2

Average Turnaround Time = 3.50
Average Waiting Time = 0.75
Average Response Time = 0.75
Process returned 0 (0x0)   execution time : 12.201 s
Press any key to continue.
```

b) SJF (Non Preemptive)

CODE:

```
#include <stdio.h>
int main() {
   int n,i,j;     int
pid[10],at[10],bt[10],ct[10],tat[10],wt[10];       int
finished[10]={0};      int
current_time=0,completed=0,min;      float
avg_tat=0,avg_wt=0;

   printf("\n\tNAME : R NANDNI\tUSN : 1BM25CS497\tSECTION : 'O'\n\n");
printf("Enter number of processes: ");     scanf("%d",&n);

   printf("Enter Arrival Time:\n");
for(i=0;i<n;i++)
   {        pid[i]=i+1;
scanf("%d",&at[i]);
   }

   printf("Enter Burst Time:\n");
for(i=0;i<n;i++)
scanf("%d",&bt[i]);


//Sorting according to arrival time     for(i=0;i<n-
1;i++)
   {
      for(j=i+1;j<n;j++)
      {
if(at[i]>at[j])
```

```
{               int
temp;

temp=at[i];
at[i]=at[j];
at[j]=temp;

temp=bt[i];
bt[i]=bt[j];
bt[j]=temp;

            temp=pid[i];
pid[i]=pid[j];              pid[j]=temp;
          }
       }
   }

//Scheduling
while(completed<n)
   {
min=-1;

       for(i=0;i<n;i++)
       {
          if(at[i]<=current_time && finished[i]==0)
          {
             if(min==-1 || bt[i]<bt[min])
                min=i;
          }
       }
```

```
        if(min==-1)
        {
            current_time++;
        }
else
        {
            ct[min]=current_time+bt[min];
tat[min]=ct[min]-at[min];
wt[min]=tat[min]-bt[min];

            current_time=ct[min];

            finished[min]=1;
completed++;

            avg_tat+=tat[min];
avg_wt+=wt[min];
        }
    }

    printf("\nSJF Non-Preemptive Scheduling\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

    for(i=0;i<n;i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }

    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f\n",avg_wt/n)
    return 0

    }
```

OUTPUT

```
Enter number of processes: 4
Enter Arrival Time:
0 8 3 5
Enter Burst Time:
7 3 4 6

SJF Non-Preemptive Scheduling
PID     AT      BT      CT      TAT     WT
P1      0       7       7       7       0
P3      3       4       11      8       4
P4      5       6       20      15      9
P2      8       3       14      6       3

Average Turnaround Time = 9.00
Average Waiting Time = 4.00

Process returned 0 (0x0)   execution time : 25.889 s
Press any key to continue.
```

c) SRTF (SJF Preemptive)

CODE:

```
#include <stdio.h>

int main() {     int
n,i,time=0,completed=0;     int
pid[10],at[10],bt[10],rt[10];     int
ct[10],tat[10],wt[10];     int
idx,min_rt;     float
avg_tat=0,avg_wt=0;

   printf("\n\tNAME : R NANDINI\tUSN : 1BM25CS497\tSECTION : 'O'\n\n");

   printf("Enter number of processes: ");
scanf("%d",&n);

   printf("Enter Arrival Time:\n");
for(i=0;i<n;i++)
   {        pid[i]=i+1;
scanf("%d",&at[i]);
   }

   printf("Enter Burst Time:\n");
for(i=0;i<n;i++)
   {
      scanf("%d",&bt[i]);
rt[i]=bt[i];
   }
```

```
   while(completed<n)
   {       idx=-1;
min_rt=9999;

       for(i=0;i<n;i++)
       {
          if(at[i]<=time && rt[i]>0 && rt[i]<min_rt)
          {
min_rt=rt[i];
idx=i;
          }
       }

       if(idx==-1)
       {
time++;        }
else       {
rt[idx]--;
time++;

          if(rt[idx]==0)
          {              ct[idx]=time;
tat[idx]=ct[idx]-at[idx];
wt[idx]=tat[idx]-bt[idx];
avg_tat+=tat[idx];
avg_wt+=wt[idx];
completed++;
          }
       }
   }
```

```
    printf("\nSJF Preemptive Scheduling (SRTF)\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

    for(i=0;i<n;i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
    }

    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);

    return 0;
}
```

OUTPUT:

```
Enter number of processes: 4
Enter Arrival Time:
0 8 3 5
Enter Burst Time:
7 3 2 6

SJF Preemptive Scheduling (SRTF)
PID     AT      BT      CT      TAT     WT
P1      0       7       9       9       2
P2      8       3       12      4       1
P3      3       2       5       2       0
P4      5       6       18      13      7

Average Turnaround Time = 7.00
Average Waiting Time = 2.50

Process returned 0 (0x0)   execution time : 55.467 s
Press any key to continue.
_
```

d) Priority (Non Preemptive)

CODE:

```
#include <stdio.h>

int main() {
   int n,i;     int
pid[10],at[10],bt[10],pr[10];      int
ct[10],tat[10],wt[10];      int
finished[10]={0};      int
current_time=0,completed=0,max;
float avg_tat=0,avg_wt=0;

   printf("\n\tNAME : R NANDINI\tUSN : 1BM25CS497\tSECTION : 'O'\n\n");

   printf("Enter number of processes: ");
scanf("%d",&n);

   printf("Enter Arrival Time:\n");
for(i=0;i<n;i++)
   {         pid[i]=i+1;
scanf("%d",&at[i]);
   }

   printf("Enter Burst Time:\n");
for(i=0;i<n;i++)
scanf("%d",&bt[i]);      printf("Enter
Priority:\n");      for(i=0;i<n;i++)
scanf("%d",&pr[i]);
```

```
   while(completed<n)
   {
max=-1;

      for(i=0;i<n;i++)
      {
         if(at[i]<=current_time && finished[i]==0)
         {
            if(max==-1 || pr[i]>pr[max])
max=i;
         }
      }

      if(max==-1)
current_time++;         else
      {
         ct[max]=current_time+bt[max];
tat[max]=ct[max]-at[max];
wt[max]=tat[max]-bt[max];

         current_time=ct[max];

         finished[max]=1;
completed++;
avg_tat+=tat[max];
avg_wt+=wt[max];
      }
   }
```

```
    printf("\nPriority Scheduling (Higher number = Higher priority)\n");
printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

    for(i=0;i<n;i++)
        {
            printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);
        }


    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);

    return 0;
}
```

OUTPUT:

```
Enter number of processes: 5
Enter Arrival Time:
0 2 3 4 6
Enter Burst Time:
3 2 5 4 1
Enter Priority:
5 3 2 4 1

Priority Scheduling (Higher number = Higher priority)
PID     AT      BT      PR      CT      TAT     WT
P1      0       3       5       3       3       0
P2      2       2       3       5       3       1
P3      3       5       2       14      11      6
P4      4       4       4       9       5       1
P5      6       1       1       15      9       8

Average Turnaround Time = 6.20
Average Waiting Time = 3.20

Process returned 0 (0x0)   execution time : 94.714 s
Press any key to continue.
```

e) Priority (Preemptive)

CODE:

```
#include <stdio.h>

int main() {
   int n,i;     int
pid[10],at[10],bt[10],pr[10],rt[10];      int
ct[10],tat[10],wt[10];      int
completed=0,current_time=0,min;
float avg_tat=0,avg_wt=0;

   printf("\n\tNAME : R NANDINI\tUSN : 1BM25CS497\tSECTION : 'O'\n\n");

   printf("Enter number of processes: ");
scanf("%d",&n);

   printf("Enter Arrival Time:\n");
for(i=0;i<n;i++)
   {        pid[i]=i+1;
scanf("%d",&at[i]);
   }

   printf("Enter Burst Time:\n");
for(i=0;i<n;i++)
   {
      scanf("%d",&bt[i]);
rt[i]=bt[i];

   }
```

```
   printf("Enter Priority:\n");
for(i=0;i<n;i++)
scanf("%d",&pr[i]);

   while(completed<n)
   {
min=-1;

      for(i=0;i<n;i++)
      {
         if(at[i]<=current_time && rt[i]>0)
         {
            if(min==-1 || pr[i] < pr[min])
min=i;
         }
      }

      if(min==-1)
current_time++;
else      {
rt[min]--;
current_time++;

         if(rt[min]==0)
         {
            ct[min]=current_time;
tat[min]=ct[min]-at[min];
wt[min]=tat[min]-bt[min];

            completed++;
```

```
            avg_tat+=tat[min];
avg_wt+=wt[min];
          }
       }
   }

   printf("\nPriority Scheduling (Preemptive) Higher the Number, Higher the Priority\n");
printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

   for(i=0;i<n;i++)
printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
      pid[i],at[i],bt[i],pr[i],ct[i],tat[i],wt[i]);

   printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);

   return 0;
}
```

OUTPUT:

```
Enter number of processes: 7
Enter Arrival Time:
0 1 3 4 5 6 10
Enter Burst Time:
8 2 4 1 6 5 1
Enter Priority:
3 4 4 5 2 6 1

Priority Scheduling (Preemptive) Higher the Number, Higher the Priority
PID     AT      BT      PR      CT      TAT     WT
P1      0       8       3       15      15      7
P2      1       2       4       17      16      14
P3      3       4       4       21      18      14
P4      4       1       5       22      18      17
P5      5       6       2       12      7       1
P6      6       5       6       27      21      16
P7      10      1       1       11      1       0

Average Turnaround Time = 13.71
Average Waiting Time = 9.86

Process returned 0 (0x0)   execution time : 54.150 s
Press any key to continue.
```

f) Round Robin

CODE:

```
#include <stdio.h>

int main() {
   int n,i;     int
pid[10],at[10],bt[10],rt[10];      int
ct[10],tat[10],wt[10];      int
tq,current_time=0,completed=0;
float avg_tat=0,avg_wt=0;

   printf("\n\tNAME : R NANDINI\tUSN : 1BM25CS497\tSECTION : 'O'\n\n");

   printf("Enter number of processes: ");
scanf("%d",&n);

   printf("Enter Arrival Time:\n");
for(i=0;i<n;i++)
   {        pid[i]=i+1;
scanf("%d",&at[i]);
   }

   printf("Enter Burst Time:\n");
for(i=0;i<n;i++)
   {
      scanf("%d",&bt[i]);
rt[i]=bt[i];
```

```
    }

    printf("Enter Time Quantum: ");
scanf("%d",&tq);

    int done;
while(completed<n)
    {
done=1;

        for(i=0;i<n;i++)
        {
            if(at[i]<=current_time && rt[i]>0)
            {
done=0;
                 if(rt[i] >
tq)
                {
current_time += tq;
rt[i] -= tq;                }
else
                {
current_time += rt[i];
rt[i]=0;
ct[i]=current_time;
tat[i]=ct[i]-at[i];
wt[i]=tat[i]-bt[i];
```

```
                completed++;

avg_tat+=tat[i];
avg_wt+=wt[i];
            }
          }
       }

       if(done==1)
       {
         current_time++;
       }

   }

   printf("\nRound Robin Scheduling\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

   for(i=0;i<n;i++)
   {
      printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
      pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);
   }
   printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);

   return 0;
}
```

OUTPUT:

```
Enter number of processes: 5
Enter Arrival Time:
0 1 2 3 4
Enter Burst Time:
5 3 1 2 3
Enter Time Quantum: 2

Round Robin Scheduling
PID     AT      BT      CT      TAT     WT
P1      0       5       14      14      9
P2      1       3       12      11      8
P3      2       1       5       3       2
P4      3       2       7       4       2
P5      4       3       13      9       6

Average Turnaround Time = 8.20
Average Waiting Time = 5.40

Process returned 0 (0x0)   execution time : 15.305 s
Press any key to continue.
```

LAB PROGRAM - 2

Question:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE:

```
#include <stdio.h>


int main() {
   int n,i;     int
at[10],bt[10],rt[10],q[10];      int
ct[10],tat[10],wt[10];      int
tq,current_time=0,completed=0;


   float avg_tat=0,avg_wt=0;


   printf("\n\tNAME : R NANDINI\tUSN : 1BM25CS497\tSECTION : 'O'\n\n");


   printf("Enter number of processes: ");
scanf("%d",&n);


   printf("Enter Arrival Time:\n");
for(i=0;i<n;i++)
   {
scanf("%d",&at[i]);
   }
```

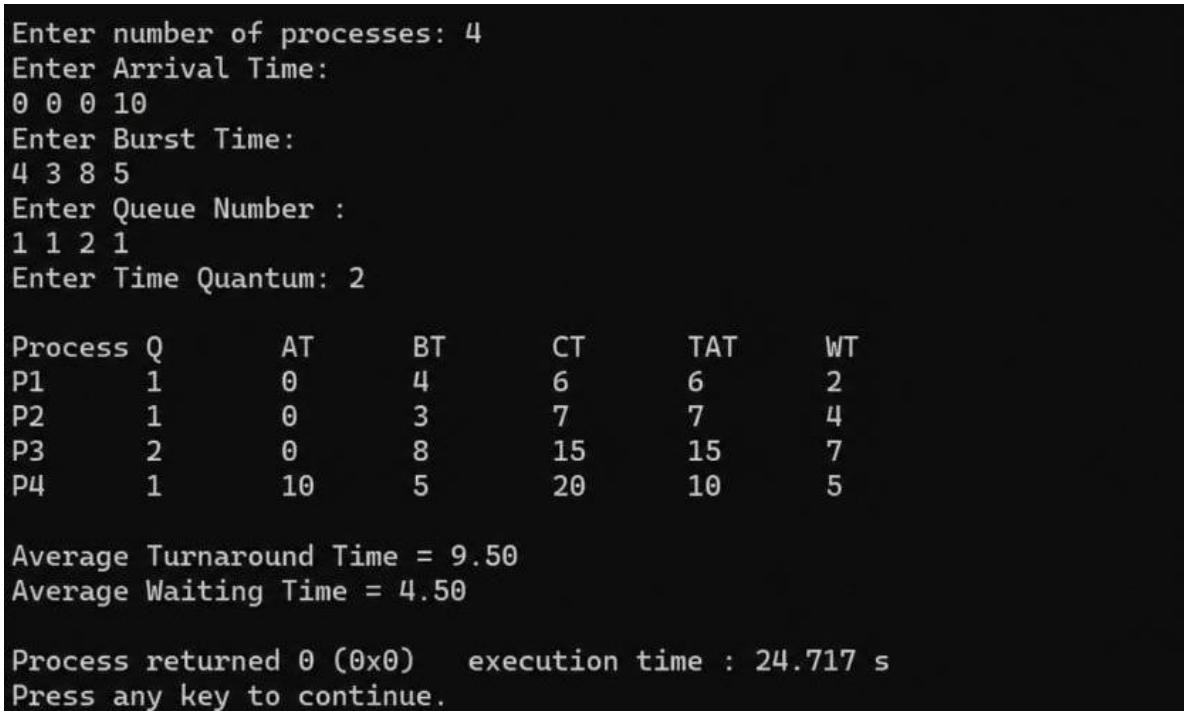
```
   printf("Enter Burst Time:\n");
for(i=0;i<n;i++)
   {
      scanf("%d",&bt[i]);
rt[i]=bt[i];
   }

   printf("Enter Queue Number :\n");
for(i=0;i<n;i++)
   {
      scanf("%d",&q[i]);
   }

   printf("Enter Time Quantum: ");
scanf("%d",&tq);


   while(completed < n)
   {       int executed
= 0;

// FIRST: QUEUE 1 (ROUND ROBIN)
      for(i=0;i<n;i++)
      {
         if(q[i]==1 && rt[i]>0 && at[i] <= current_time)
         {
executed = 1;
if(rt[i] > tq)
```

```
            {
current_time += tq;
rt[i] -= tq;
            }
else
            {
current_time += rt[i];
rt[i] = 0;

                ct[i] = current_time;
completed++;
            }
        }
    }

    if(executed == 0)
    {
        // SECOND: QUEUE 2 (FCFS)

        for(i=0;i<n;i++)
        {
            if(q[i]==2 && rt[i]>0 && at[i] <= current_time)
            {
current_time += rt[i];
rt[i] = 0;

                ct[i] = current_time;
completed++;
```

```
                executed =
1;                  break;
            }
        }
    }


    if(executed == 0)
current_time++;
  }

// calculating average TAT and WT
for(i=0;i<n;i++)
  {       tat[i] = ct[i] -
at[i];       wt[i]  = tat[i] -
bt[i];

    avg_tat += tat[i];
avg_wt += wt[i];
  }

  printf("\nProcess\tQ\tAT\tBT\tCT\tTAT\tWT\n");

  for(i=0;i<n;i++)
  {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
    i+1,q[i],at[i],bt[i],ct[i],tat[i],wt[i]);
  }
```

```
    printf("\nAverage Turnaround Time = %.2f",avg_tat/n);
printf("\nAverage Waiting Time = %.2f\n",avg_wt/n);


    return 0;
}
```

OUTPUT:



LAB PROGRAM – 3

Question:

Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic
b) Earliest-deadline First
c) Proportional scheduling

a) Rate- Monotonic

CODE:

```
#include <stdio.h> #include
<math.h>
//TO CALCULATE GCD
int gcd(int a, int b)
{    return (b == 0) ? a : gcd(b, a %
b);
}
//TO CALCULATE LCM
int lcm(int a, int b)
{    return (a * b) / gcd(a,
b);
}

int main() {
   int n, i;    int bt[10],
pt[10], rt[10];    int hyper,
time = 0;

   printf("\n\tNAME : R NANDINI\tUSN : 1BM25CS497\tSECTION : 'O'\n\n");

   printf("Enter number of processes: ");
scanf("%d", &n);

   printf("Enter Burst Times:\n");
for (i = 0; i < n; i++)
      {
         scanf("%d", &bt[i]);

         rt[i] = 0; // initially no job released
      }
```

```
   printf("Enter Periods:\n");
for (i = 0; i < n; i++)
      {
         scanf("%d", &pt[i]);
      }

// Calculate hyperperiod = LCM of all periods
hyper = pt[0];     for (i = 1; i < n; i++)
      {
         hyper = lcm(hyper, pt[i]);
      }

   printf("LCM=%d\n\n", hyper);

   printf("Rate Monotone Scheduling:\n");
printf("PID   Burst   Period\n");     for (i =
0; i < n; i++)
      {           printf("%d     %d      %d\n", i+1,
bt[i], pt[i]);
      }

// Utilization check
double U = 0;     for (i
= 0; i < n; i++)
      {
         U += (double)bt[i] / pt[i];
      }
   double bound = n * (pow(2.0, 1.0/n) - 1);
   printf("\n%.6f <= %.6f => %s\n", U, bound, (U <= bound) ? "true" : "false");
printf("Scheduling occurs for %d ms\n\n", hyper);

// Timeline simulation (block-style)
int current_running = -1;     while
(time < hyper)
      {
// Release jobs        for (i = 0;
i < n; i++) {           if (time %
pt[i] == 0) {              rt[i] =
bt[i];
         }
      }

// Pick highest priority (smallest period)
int min_period = 9999, index = -1;        for
(i = 0; i < n; i++) {           if (rt[i] > 0 &&
```

```
pt[i] < min_period) {                min_period
= pt[i];              index = i;
            }
        }
        // Print only when event changes
if (index != current_running)
            {
            if (index != -1)
            {
               printf("%dms onwards: Process %d running\n", time, index+1);
            }
else
            {
               printf("%dms onwards: CPU is idle\n", time);
            }
            current_running = index;
            }
        // Execute one unit if a process is running
if (index != -1)
            {
               rt[index]--;
            }

        time++;
    }

    return 0;
}
```

OUTPUT:

```
Enter number of processes: 2
Enter Burst Times:
20 35
Enter Periods:
50 100
LCM=100

Rate Monotone Scheduling:
PID   Burst   Period
1     20      50
2     35      100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

Process returned 0 (0x0)   execution time : 14.917 s
Press any key to continue.
```

b) Earliest-deadline First

CODE:

```
#include <stdio.h>

int main() {
   int n, i;

   printf("\n\tNAME : R NANDINI\tUSN : 1BM25CS497\tSECTION : 'O'\n\n");
   printf("Enter number of tasks: ");
scanf("%d", &n);     int bt[n], dl[n], p[n],
rem[n];     printf("Enter Burst times:\n");
for (i = 0; i < n; i++) {         scanf("%d",
&bt[i]);         rem[i] = 0; // initially no job
released
   }

   printf("Enter Deadlines:\n");
for (i = 0; i < n; i++) {
scanf("%d", &dl[i]);
   }

   printf("Enter Periods:\n");
for (i = 0; i < n; i++) {
scanf("%d", &p[i]);
   }

   printf("\nPID\tBurst\tDeadline\tPeriod\n");  for (i = 0;
i < n; i++) {     printf("%d\t%d\t%d\t\t%d\n", i + 1, bt[i],
dl[i], p[i]);
```

```
    }

    int total_time = p[0];
for (i = 1; i < n; i++) {
if (p[i] > total_time)
total_time = p[i];
    }

    printf("\nScheduling occurs for %d ms\n\n", total_time);

    int prev_idle = 0;



    for (int time = 0; time < total_time; time++) {
for (i = 0; i < n; i++) {             if (time % p[i]
== 0) {                 rem[i] = bt[i];
            }
        }

// Select task with earliest deadline
int selected = -1;         int
min_deadline = 9999;         for (i =
0; i < n; i++) {             if (rem[i] >
0) {                 if (dl[i] <
min_deadline) {
min_deadline = dl[i];
selected = i;

                }
            }
        }
```

```
// Print timeline        if (selected == -1) {            if
(!prev_idle) {               printf("%dms onwards: CPU is
idle.\n", time);               prev_idle = 1;
          }
      } else {           printf("%dms onwards: Task %d is running.\n", time,
selected + 1);           rem[selected]--;           prev_idle = 0;
      }
   }

   return 0;
}
```

OUTPUT:

```
PID     Burst   Deadline        Period
1       3       7               20
2       2       4               5
3       2       8               10

Scheduling occurs for 20 ms

0ms onwards: Task 2 is running.
1ms onwards: Task 2 is running.
2ms onwards: Task 1 is running.
3ms onwards: Task 1 is running.
4ms onwards: Task 1 is running.
5ms onwards: Task 2 is running.
6ms onwards: Task 2 is running.
7ms onwards: Task 3 is running.
8ms onwards: Task 3 is running.
9ms onwards: CPU is idle.
10ms onwards: Task 2 is running.
11ms onwards: Task 2 is running.
12ms onwards: Task 3 is running.
13ms onwards: Task 3 is running.
14ms onwards: CPU is idle.
15ms onwards: Task 2 is running.
16ms onwards: Task 2 is running.
17ms onwards: CPU is idle.

Process returned 0 (0x0)   execution time : 33.521 s
Press any key to continue.
```

c) Proportional scheduling

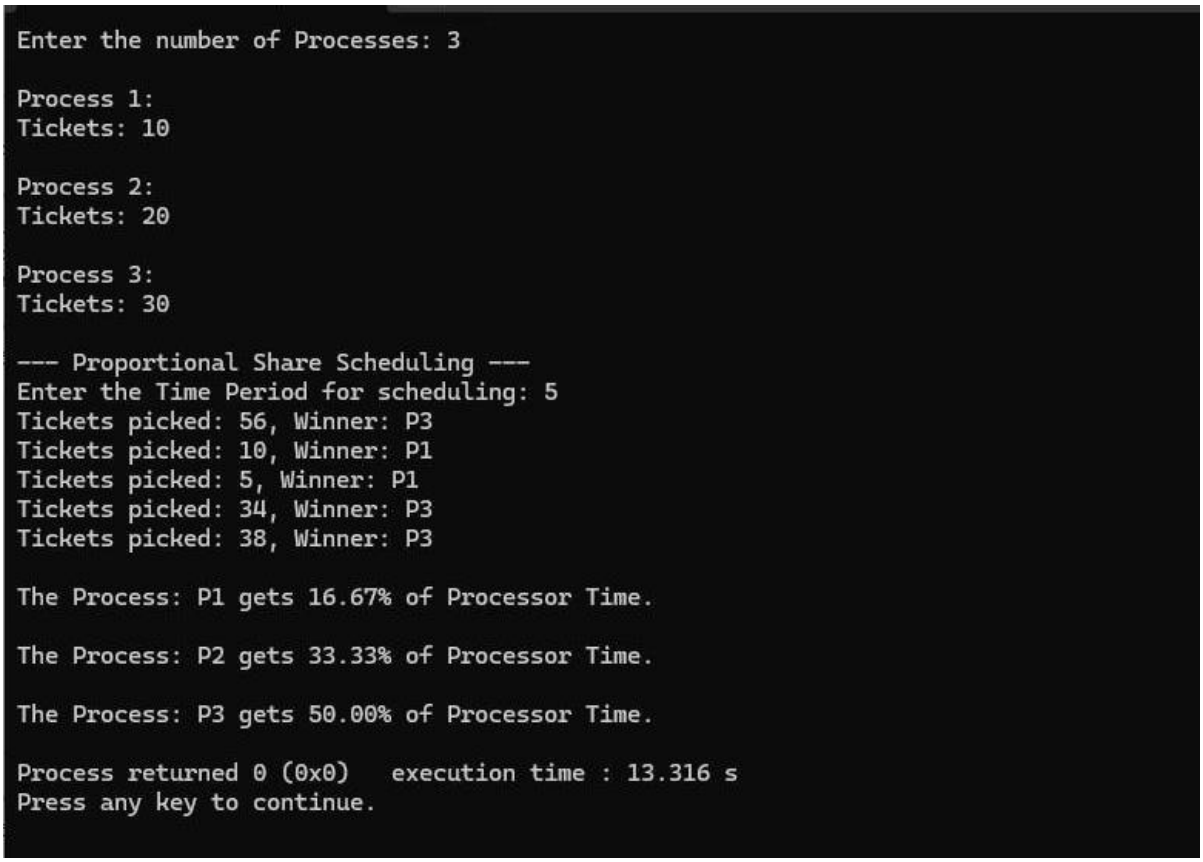
CODE:

```
// Proportional Share Scheduling (Lottery Scheduling)

#include  <stdio.h>
#include <stdlib.h>
#include <time.h>

// Define Process structure
typedef struct {     char
name[5];     int tickets;
} Process;

int main() {     int n,
total_tickets = 0;     float
total_T = 0.0;

   printf("Enter the number of Processes: ");
scanf("%d", &n);

   Process p[n];     srand(time(NULL)); // Seed random
number generator

   // Input process details      for (int i
= 0; i < n; i++) {
printf("\nProcess %d:\n", i + 1);
sprintf(p[i].name, "P%d", i + 1);
printf("Tickets: ");         scanf("%d",
&p[i].tickets);
```

```
        total_tickets += p[i].tickets;
total_T += p[i].tickets;
    }

    printf("\n--- Proportional Share Scheduling ---\n");

    // Input scheduling time period     int m;
printf("Enter the Time Period for scheduling: ");
scanf("%d", &m);

    // Lottery scheduling simulation      for (int i = 0; i
< m; i++) {         int winning_ticket = rand() %
total_tickets + 1;         int accumulated_tickets = 0;
int winner_index = -1;

        for (int j = 0; j < n; j++) {
accumulated_tickets += p[j].tickets;             if
(winning_ticket <= accumulated_tickets) {
winner_index = j;                 break;
            }
        }

        printf("Tickets picked: %d, Winner: %s\n",
winning_ticket, p[winner_index].name);     }

    // Print proportional share of CPU time      for (int i = 0; i < n;
i++) {         printf("\nThe Process: %s gets %.2f%% of Processor
Time.\n",                 p[i].name, ((p[i].tickets / total_T) * 100));     }
```

```
    return 0;
}
```

OUTPUT:



LAB PROGRAM – 4

Question:

Write a C program to simulate: (Any one)

a) Producer-Consumer problem using semaphores.

b) Dining-Philosopher's problem

a) Producer-Consumer problem using semaphores.

CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <semaphore.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty;   // counts empty slots sem_t full;
// counts filled slots pthread_mutex_t mutex; //
for mutual exclusion

// Producer function void*
producer(void* arg) {
   int item, i = 0;     while (1) {         item = i++; // produce an item
(here just incrementing number)         sem_wait(&empty);
// wait for empty slot         pthread_mutex_lock(&mutex);     // enter
critical section
```

```
        buffer[in] = item;          printf("Produced: %d at
buffer[%d]\n", item, in);          in = (in + 1) %
BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);  // leave critical section
sem_post(&full);                // signal that new item is available

        sleep(1); // simulate time to produce
    }
    return NULL;
}

// Consumer function void*
consumer(void* arg) {
    int item;     while (1) {       sem_wait(&full);                //
wait for filled slot    pthread_mutex_lock(&mutex);  // enter
critical section

        item = buffer[out];         printf("Consumed: %d from
buffer[%d]\n", item, out);         out = (out + 1) %
BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);  // leave critical section
sem_post(&empty);               // signal that a slot is free

        sleep(2); // simulate time to consume
    }
    return NULL;
}
```

```
int main() {
pthread_t prod, cons;

    // Initialize semaphores and mutex
sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
pthread_mutex_init(&mutex, NULL);

    // Create producer and consumer threads
pthread_create(&prod, NULL, producer, NULL);
pthread_create(&cons, NULL, consumer, NULL);

    // Wait for threads to finish (they won't in this infinite loop)
pthread_join(prod, NULL);     pthread_join(cons, NULL);

    // Cleanup (not reached in this infinite loop)
sem_destroy(&empty);     sem_destroy(&full);
pthread_mutex_destroy(&mutex);

    return 0;
}
```

OUTPUT:

```
Produced: 0 at buffer[0]
Consumed: 0 from buffer[0]
Produced: 1 at buffer[1]
Produced: 2 at buffer[2]
Consumed: 1 from buffer[1]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Produced: 7 at buffer[2]
Consumed: 4 from buffer[4]
Produced: 8 at buffer[3]
Produced: 9 at buffer[4]
Consumed: 5 from buffer[0]
Produced: 10 at buffer[0]
Consumed: 6 from buffer[1]
Produced: 11 at buffer[1]
Consumed: 7 from buffer[2]
Produced: 12 at buffer[2]
Consumed: 8 from buffer[3]
Produced: 13 at buffer[3]
Consumed: 9 from buffer[4]
Produced: 14 at buffer[4]
Consumed: 10 from buffer[0]
Produced: 15 at buffer[0]
Consumed: 11 from buffer[1]
Produced: 16 at buffer[1]
Consumed: 12 from buffer[2]
Produced: 17 at buffer[2]
Consumed: 13 from buffer[3]
Produced: 18 at buffer[3]
Consumed: 14 from buffer[4]
Produced: 19 at buffer[4]
Consumed: 15 from buffer[0]
Produced: 20 at buffer[0]
Consumed: 16 from buffer[1]
Produced: 21 at buffer[1]
Consumed: 17 from buffer[2]
Produced: 22 at buffer[2]
Consumed: 18 from buffer[3]
Produced: 23 at buffer[3]
Consumed: 19 from buffer[4]
Produced: 24 at buffer[4]
Consumed: 20 from buffer[0]
Produced: 25 at buffer[0]
Consumed: 21 from buffer[1]
Produced: 26 at buffer[1]
Consumed: 22 from buffer[2]
Produced: 27 at buffer[2]
```

b) Dining-Philosopher’s problem

CODE:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define N 5 // Number of philosophers

pthread_mutex_t forks[N];       // One mutex per fork pthread_t
philosophers[N];      // Philosopher threads

// Philosopher routine void*
philosopher(void* num) {     int id =
*(int*)num;     int left = id;              // left fork
index    int right = (id + 1) % N;  // right fork
index

   while (1) {         // Thinking
printf("Philosopher %d is thinking.\n", id);
sleep(1);

      // To avoid deadlock:
      // Even philosophers pick left then right         // Odd
philosophers pick right then left         if (id % 2 == 0) {
pthread_mutex_lock(&forks[left]);              printf("Philosopher %d
picked up left fork %d.\n", id, left);
pthread_mutex_lock(&forks[right]);              printf("Philosopher
%d picked up right fork %d.\n", id, right);
```

```
        } else {            pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked up right fork %d.\n", id, right);

            pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked up left fork %d.\n", id, left);
        }

        // Eating         printf("Philosopher %d is
eating.\n", id);         sleep(2);

        // Put down forks         pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);          printf("Philosopher %d put
down forks %d and %d.\n", id, left, right);
    }

    return NULL;
}

int main() {
    int i;
int ids[N];

    // Initialize forks (mutexes)      for (i = 0; i
< N; i++) {
pthread_mutex_init(&forks[i], NULL);
ids[i] = i;
    }

    // Create philosopher threads      for (i = 0; i < N; i++) {
pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
```

```
    }

    // Join threads (infinite loop, but good practice)
for (i = 0; i < N; i++) {
pthread_join(philosophers[i], NULL);
    }

    // Destroy mutexes (unreachable here)
for (i = 0; i < N; i++) {
pthread_mutex_destroy(&forks[i]);
    }

    return 0;
}
```

OUTPUT:

```
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 0 picked up left fork 0.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 1 is thinking.
Philosopher 2 picked up left fork 2.
Philosopher 3 put down forks 3 and 4.
Philosopher 4 picked up left fork 4.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 3 is thinking.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 1 picked up right fork 2.
Philosopher 1 picked up left fork 1.
Philosopher 1 is eating.
Philosopher 2 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 4 put down forks 4 and 0.
Philosopher 4 is thinking.
Philosopher 0 picked up left fork 0.
Philosopher 0 picked up right fork 1.
Philosopher 0 is eating.
Philosopher 2 picked up left fork 2.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 4 picked up left fork 4.
Philosopher 2 picked up right fork 3.
Philosopher 2 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 4 picked up right fork 0.
Philosopher 4 is eating.
```

LAB PROGRAM – 5

Question:

Write a C program to simulate:
a) Bankers' algorithm for the purpose of deadlock avoidance.
b) Deadlock Detection

a) Bankers' algorithm for the purpose of deadlock avoidance.

CODE:

```
#include <stdio.h>


int main()
{     printf("\n\tNAME : R NANDINI\tUSN : 1BM25CS497\tSECTION :
'O'\n\n");
   int n, m, i, j, k;


   printf("Enter number of processes: ");
scanf("%d", &n);


   printf("Enter number of resources: ");
scanf("%d", &m);


   int alloc[n][m], max[n][m], need[n][m];
int avail[m];


   printf("Enter Allocation Matrix:\n");
for(i = 0; i < n; i++)
   {         for(j = 0; j < m;
j++)
       {
          scanf("%d", &alloc[i][j]);
```

```
        }
    }

    printf("Enter Maximum Demand Matrix:\n");
    for(i = 0; i < n; i++)
    {       for(j = 0; j < m;
j++)
        {
            scanf("%d", &max[i][j]);
        }
    }

    printf("Enter Available Resources:\n");
for(i = 0; i < m; i++)
    {       scanf("%d",
&avail[i]);
    }

    for(i = 0; i < n; i++)
    {       for(j = 0; j < m;
j++)
        {           need[i][j] = max[i][j] -
alloc[i][j];
        }
    }

    int finish[n], safeSeq[n], work[m];

    for(i = 0; i < n; i++)
```

```
    {       finish[i]
= 0;
    }

    for(i = 0; i < m; i++)
    {       work[i] =
avail[i];
    }

    int count = 0;

    while(count < n)
    {       int found
= 0;

        for(i = 0; i < n; i++)
        {           if(finish[i]
== 0)           {
int possible = 1;

                for(j = 0; j < m; j++)
                {
if(need[i][j] > work[j])
                    {
possible = 0;
break;
                    }
                }

                if(possible)
```

```
            {                   for(k = 0;
k < m; k++)
                {                       work[k]
+= alloc[i][k];
                }

                safeSeq[count++] = i;
finish[i] = 1;                  found =
1;
            }
          }
       }

       if(found == 0)
       {
break;
       }
   }

   if(count == n)
   {              printf("\nSystem is in a safe
state.\n");        printf("Safe sequence is: ");

       for(i = 0; i < n; i++)
       {
          printf("P%d", safeSeq[i]);
           if(i != n -
1)            {
printf(" -> ");
          }
```

```
        }
}
else
   {
      printf("\nSystem is NOT in a safe state.");
   }


   return 0;
}
```

OUTPUT:

```
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2

Process returned 0 (0x0)   execution time : 52.958 s
Press any key to continue.
```

b) Deadlock Detection

CODE:

```
#include <stdio.h>

int main()
{    printf("\n\tNAME : R NANDINI\tUSN : 1BM25CS497\tSECTION :
'O'\n\n");
   int n, m, i, j, k;

   printf("Enter the number of processes: ");
scanf("%d", &n);

   printf("Enter the number of resources: ");
scanf("%d", &m);

   int alloc[n][m], request[n][m];
int avail[m];

   printf("Enter the allocation matrix:\n");

   for(i = 0; i < n; i++)
   {        printf("Process %d:
", i);

      for(j = 0; j < m; j++)
      {
         scanf("%d", &alloc[i][j]);
      }
   }
```

```
    printf("Enter the request matrix:\n");

    for(i = 0; i < n; i++)
    {       printf("Process %d:
", i);

        for(j = 0; j < m; j++)
        {
            scanf("%d", &request[i][j]);
        }
    }

    printf("Enter the available resources: ");

    for(i = 0; i < m; i++)
    {       scanf("%d",
&avail[i]);
    }

    int finish[n], safeSeq[n], work[m];

    for(i = 0; i < m; i++)
    {       work[i] =
avail[i];
    }

    for(i = 0; i < n; i++)
    {       finish[i]
= 0;    }
```

```
    int count = 0;

    while(count < n)
    {       int found
= 0;

        for(i = 0; i < n; i++)
        {           if(finish[i]
== 0)           {
int possible = 1;

            for(j = 0; j < m; j++)
            {
if(request[i][j] > work[j])
                {
possible = 0;
break;
                }
            }
if(possible)
            {               for(k = 0;
k < m; k++)
                {                   work[k]
+= alloc[i][k];
                }

                safeSeq[count++] = i;
finish[i] = 1;                  found =
1;
            }
```

```
            }
        }

        if(found == 0)
        {
break;
        }
    }

    if(count == n)
    {               printf("\nSystem is in safe
state.\n");          printf("Safe Sequence is:
");

        for(i = 0; i < n; i++)
        {
            printf("P%d ", safeSeq[i]);
        }
}
else
    {
        printf("\nDeadlock detected.\n");
    }

    return 0;

 }
```

OUTPUT:

```
Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0

System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1

Process returned 0 (0x0)   execution time : 61.305 s
Press any key to continue.
```

LAB PROGRAM – 6

Question :

Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit
b) Best-fit
c) First-fit

CODE:

```
#include <stdio.h>


int main()
{     printf("\n\tNAME : R NANDINI\tUSN : 1BM25CS497\tSECTION :
'O'\n\n");
   int b[20], p[20];
int temp[20];     int
i, j, n, m;     int
allocation[20];


   printf("Enter number of memory blocks: ");
scanf("%d", &n);


   printf("Enter sizes of %d memory blocks:\n", n);
for(i = 0; i < n; i++)
   {        scanf("%d",
&b[i]);
   }


   printf("Enter number of processes: ");
scanf("%d", &m);
```

```
   printf("Enter sizes of %d processes:\n", m);
for(i = 0; i < m; i++)
   {        scanf("%d",
&p[i]);
   }

   // FIRST FIT

   for(i = 0; i < n; i++)
temp[i] = b[i];

   for(i = 0; i < m; i++)
allocation[i] = -1;

   for(i = 0; i < m; i++)
   {        for(j = 0; j < n;
j++)
       {            if(temp[j]
>= p[i])
           {
allocation[i] = j;
temp[j] = -1;
break;
           }
       }
   }

   printf("\n--- First Fit ---\n");    printf("Process
No.\tProcess Size\tBlock No.\n");    for(i = 0; i <
m; i++)
```

```
    {       printf("%d\t\t%d\t\t", i + 1,
p[i]);

      if(allocation[i]          !=          -1)
printf("%d\n", allocation[i] + 1);   else
printf("Not Allocated\n");
    }

    // BEST FIT

    for(i = 0; i < n; i++)
temp[i] = b[i];

    for(i = 0; i < m; i++)
allocation[i] = -1;

    for(i = 0; i < m; i++)
    {       int best
= -1;

      for(j = 0; j < n; j++)
      {           if(temp[j]
>= p[i])
          {
            if(best == -1 || temp[j] < temp[best])
            {
best = j;
            }
          }
      }
```

```
        if(best != -1)        {
allocation[i] = best;
temp[best] = -1;
        }
    }

    printf("\n--- Best Fit ---\n");     printf("Process
No.\tProcess Size\tBlock No.\n");

    for(i = 0; i < m; i++)
    {        printf("%d\t\t%d\t\t", i + 1,
p[i]);

        if(allocation[i]            !=            -1)
printf("%d\n", allocation[i] + 1);    else
printf("Not Allocated\n");
    }

    // WORST FIT

    for(i = 0; i < n; i++)
temp[i] = b[i];

    for(i = 0; i < m; i++)
allocation[i] = -1;     for(i = 0; i <
m; i++)
    {        int worst
= -1;
```

```
        for(j = 0; j < n; j++)
        {               if(temp[j]
>= p[i])
            {
                if(worst == -1 || temp[j] > temp[worst])
                {
worst = j;
                }
            }
        }


        if(worst != -1)
        {               allocation[i] =
worst;              temp[worst] =
-1;
        }
    }

    printf("\n--- Worst Fit ---\n");     printf("Process
No.\tProcess Size\tBlock No.\n");

    for(i = 0; i < m; i++)
    {         printf("%d\t\t%d\t\t", i + 1,
p[i]);          if(allocation[i] != -1)
printf("%d\n", allocation[i] + 1);
else              printf("Not Allocated\n");
    }

    return 0;
}
```

OUTPUT:

```
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---
Process No.     Process Size    Block No.
1               212             2
2               417             5
3               112             3
4               426             Not Allocated

--- Best Fit ---
Process No.     Process Size    Block No.
1               212             4
2               417             2
3               112             3
4               426             5

--- Worst Fit ---
Process No.     Process Size    Block No.
1               212             5
2               417             2
3               112             4
4               426             Not Allocated

Process returned 0 (0x0)   execution time : 51.544 s
Press any key to continue.
```

LAB PROGRAM – 7

Question:

Write a C program to simulate page replacement algorithms. a) FIFO
b) LRU
c) Optimal

CODE:

```
#include <stdio.h>


int main()
{    printf("\n\tNAME : R NANDINI\tUSN : 1BM25CS497\tSECTION :
'O'\n\n");
   int pages[50], frames[10];
   int n, f;
int i, j, k;


   printf("Enter number of pages: ");
scanf("%d", &n);


   printf("Enter page reference string:\n");
for(i = 0; i < n; i++)
   {
      scanf("%d", &pages[i]);
   }


   printf("Enter number of frames: ");
scanf("%d", &f);


   // - - - - - - - - - - FIFO ----------
```

```
    int faults = 0;
int next = 0;     int
found;

    for(i = 0; i < f; i++)
    {         frames[i]
= -1;
    }

    printf("\n--- FIFO Page Replacement ---\n");

    for(i = 0; i < n; i++)
    {         found
= 0;

        for(j = 0; j < f; j++)
        {             if(frames[j] ==
pages[i])
            {
found = 1;
break;
            }
        }

        if(found == 0)
        {
            frames[next] = pages[i];
next = (next + 1) % f;
faults++;
```

```
        }

        printf("Page %d -> [", pages[i]);

        for(j = 0; j < f; j++)
        {
if(frames[j] != -1)
            {               printf("%d",
frames[j]);
                 if(j != f - 1)
{                    printf("
");
               }
            }
        }

        printf("]\n");
    }

    printf("Total Page Faults (FIFO): %d\n", faults);

    // - - - - - - - - - - - LRU ----------

    int time[10];
int count = 0;
faults = 0;

    for(i = 0; i < f; i++)
    {
```

```
        frames[i] = -1;
time[i] = 0;
    }

    printf("\n--- LRU Page Replacement ---\n");

    for(i = 0; i < n; i++)
    {       found
= 0;

        for(j = 0; j < f; j++)
        {           if(frames[j] ==
pages[i])
            {               found
= 1;               count++;
time[j] = count;
break;
            }
        }

        if(found == 0)
        {           int
pos = -1;

            for(j = 0; j < f; j++)
            {
if(frames[j] == -1)
                {
pos = j;
break;
```

```
            }
        }

        if(pos == -1)
        {
pos = 0;
            for(j = 1; j < f;
j++)
            {               if(time[j]
< time[pos])
                {
pos = j;
                }
            }
        }

        frames[pos] = pages[i];
count++;          time[pos] =
count;          faults++;
    }

    printf("Page %d -> [", pages[i]);

    for(j = 0; j < f; j++)
    {
if(frames[j] != -1)
        {               printf("%d",
frames[j]);               if(j != f - 1)
{               printf(" ");
            }
```

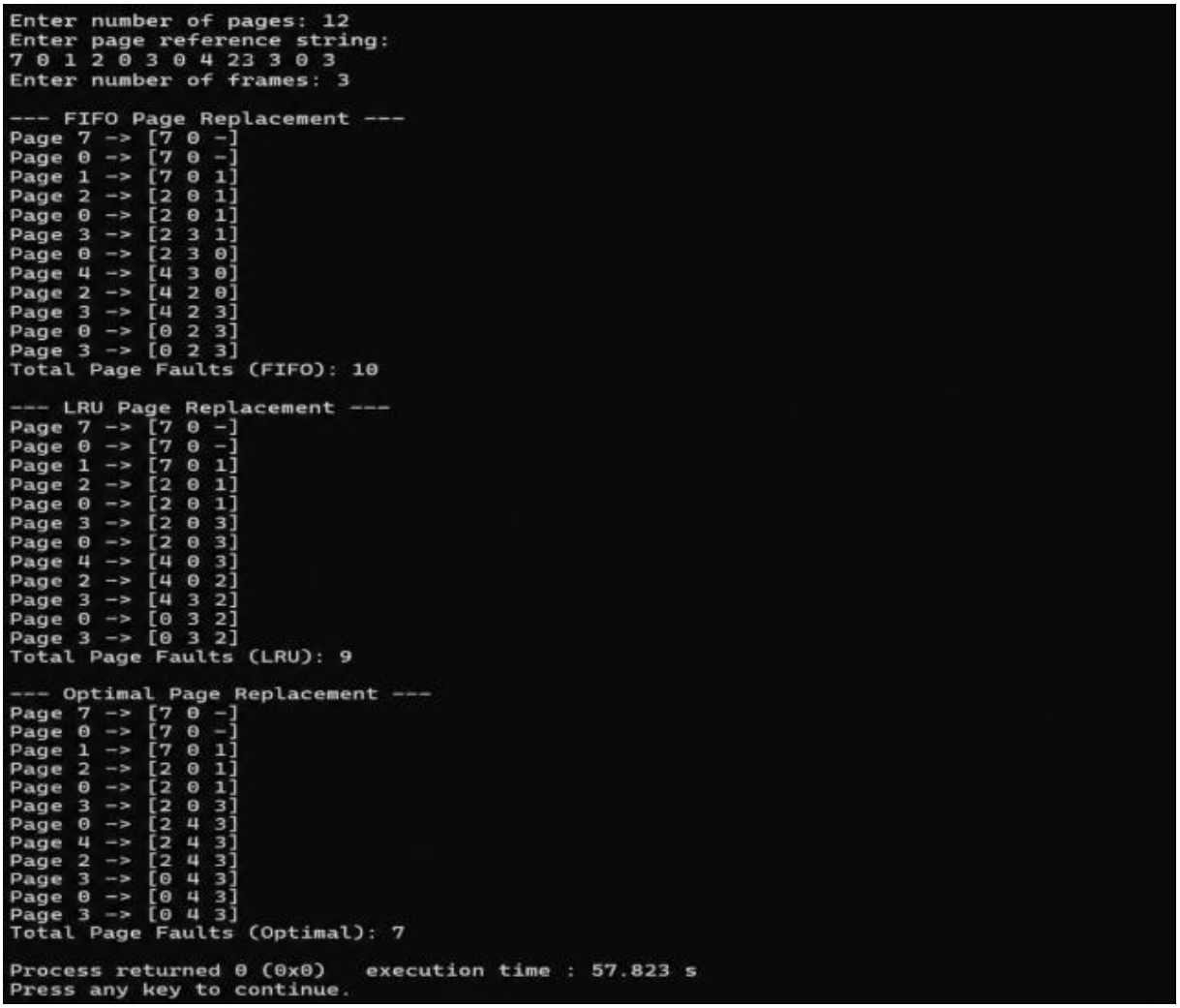
```
            }
        }

        printf("]\n");
    }

    printf("Total Page Faults (LRU): %d\n", faults);

    // ---------- OPTIMAL ----------

    faults = 0;

    for(i = 0; i < f; i++)
    {       frames[i]
= -1;
    }

    printf("\n--- Optimal Page Replacement ---\n");

    for(i = 0; i < n; i++)
    {       found
= 0;

        for(j = 0; j < f; j++)
        {
            if(frames[j] == pages[i])
            {
found = 1;
break;
            }
```

```
        }

        if(found == 0)
        {           int
pos = -1;

            for(j = 0; j < f; j++)
            {
if(frames[j] == -1)
                {
pos = j;
break;
                }
            }

            if(pos == -1)
            {               int
farthest = -1;              int
index;
                for(j = 0; j < f;
j++)
                {               int
found_future = 0;

                    for(k = i + 1; k < n; k++)
{                       if(frames[j] == pages[k])
                        {
if(k > farthest)
{                               farthest =
k;                              pos = j;
```

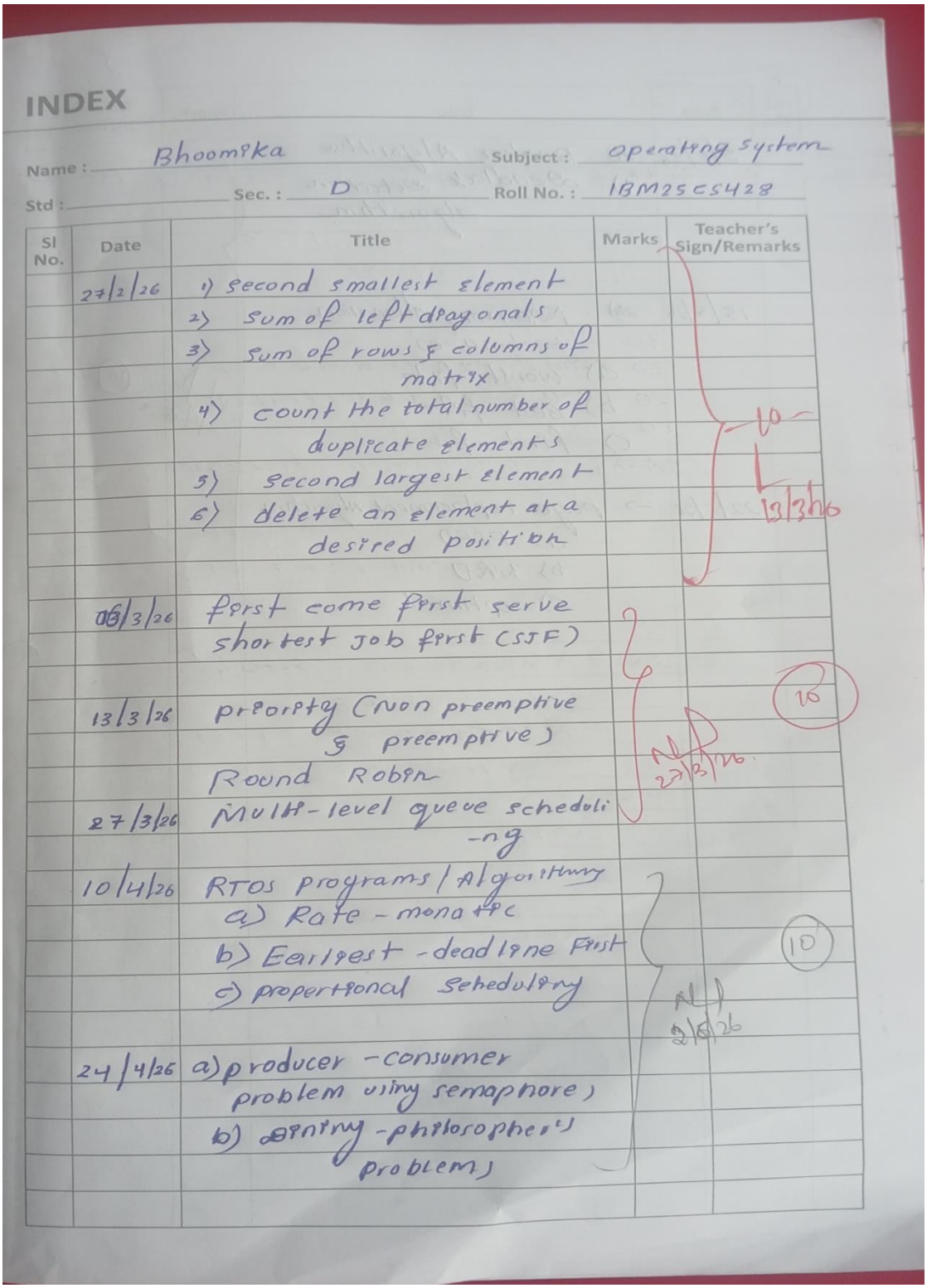
```
                    }
                    found_future =
1;                    break;
                }
            }

            if(found_future == 0)
            {
pos = j;
break;
            }
        }
    }

    frames[pos] = pages[i];
faults++;
    }

    printf("Page %d -> [", pages[i]);

    for(j = 0; j < f; j++)
    {
        if(frames[j] != -1)
        {           printf("%d",
frames[j]);
            if(j != f - 1)
{               printf("
");
        }
    }
```

```
        }

        printf("]\n");
    }

    printf("Total Page Faults (Optimal): %d\n", faults);

    return 0;
}
```

OUTPUT:



INDEX PAGE OF OBSERVATION BOOK



INDEX

Name : Bhoomika Subject : operating system

Std : Sec. : D Roll No. : 1BM25CS428

Sl No.	Date	Title	Marks	Teacher's Sign/Remarks
	27/2/26	1) second smallest element		
		2) sum of left diagonals		
		3) sum of rows & columns of matrix		
		4) count the total number of duplicate elements		
		5) second largest element		
		6) delete an element at a desired position		10 13/3/26
	06/3/26	first come first serve shortest job first (SJF)		
	13/3/26	preority (non preemptive & preemptive)		10 AL 27/3/26
		Round Robin		
	27/3/26	Multi-level queue scheduling		
	10/4/26	RTOS programs / Algorithms a) Rate-monotic		
		b) Earliest-deadline first		10
		c) proportional scheduling		AL 2/4/26
	24/4/26	a) producer-consumer problem using semaphore)		
		b) dining-philosopher's problem)		

Sl No.	Date	Title	Marks	Teacher's Sign/Remarks
	8/5/26	a) Banker Algorithm b) Deadlock detection algorithm		
	15/5/26	Memory allocation techniques. a) worst-fit b) Best-fit c) First-fit	10 2/6/26	
	22/5/26	-> Page replacement algorithm a) FIFO b) LRU c) optimal		